REVERSE ENGINEERING BLUEPRINT

CAMPUS PRINT HUB

Source Code & Architecture Blueprint Guide

Campus Print Hub: Source Code & Architectural Reverse-Engineering Guide (V3)

Welcome to the ultimate reverse-engineering manual for the **Campus Print Hub**. If you have a background in C, C++, Java, or Python, this guide is designed to help you understand how high-performance, real-time client-server web applications are constructed.

We will map concepts from traditional console/desktop programming to web architectures, explain the complete data flows, and analyze every major component and code line in this project.

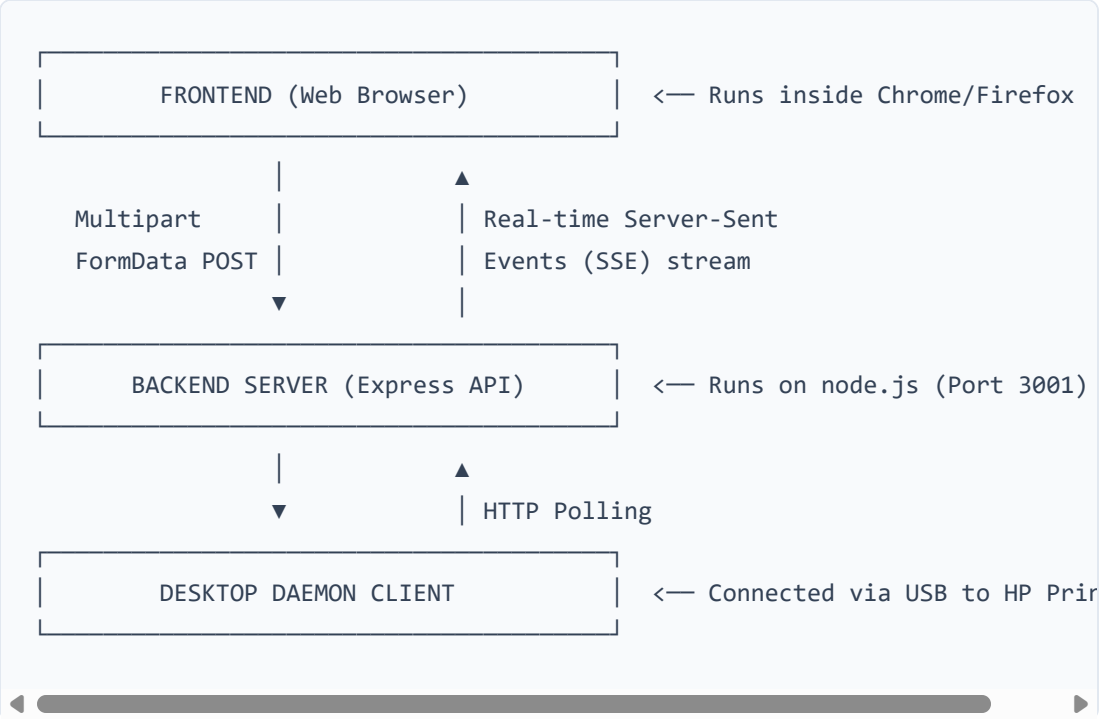
---

1. Architectural Transition: Console to Client-Server

In console programming (like a standard C++ console app), your execution is **local**, **single-threaded**, and **synchronous**:

```
[ Code starts at main() ] ——> [ Sequential Logic / Loops ] ——> [ Wait for
```

In a web application, execution is **distributed across three distinct machines** communicating asynchronously over the network using HTTP protocols and streams:



1. **Frontend (React/Vite)**: Standard HTML/CSS/TypeScript GUI interface. Runs locally in the browser to collect files and configuration choices.

2. **Backend (Node.js/Express)**: A multi-threaded web API that handles database interactions, validates binary magic bytes to check upload safety, and handles streaming connections.

3. **Desktop Daemon Client (`client.cjs`)**: A background service that runs on the print operator's PC. It polls the server, downloads raw files, and invokes Windows printing routines to send jobs to the physical printer.

---

2. Global Code Structure & Dependencies

Here is a summary of our project's packages and libraries, mapped to traditional programming equivalents:

Library/Tool	Core Responsibility	Equivalent in C++ / Java / Python
:---	:---	:---
Node.js	JavaScript runtime environment outside the browser.	Java Virtual Machine (JVM) or Python Interpreter.
Express	Lightweight HTTP routing library for handling API requests.	Python Flask / Java Spring Boot.
Vite	Bundler and compiler that builds our TypeScript into single files.	GCC Compiler (with Makefile) / CMake.
Tailwind CSS v4	Utility-first CSS styling sheet engine.	Standard UI styling APIs (e.g. JavaFX styles).
pdf-lib	Binary PDF reader to count PDF pages directly in JS.	Python PyPDF2 / Java PDFBox.
Multer	Parser middleware that processes multi-part form data uploads.	Binary Socket streams.
SumatraPDF	CLI-based PDF rendering executable for silent printing.	Command line print tools.

3. The Database Layer (`server/db.ts`)

Instead of using a database engine (like SQL), we built a **JSON file database** in `server/db.ts`. This is perfect for high-speed, lightweight operations.

Key Code Blocks:

- **Database Schema (`DbJob`)**:

```
export interface DbJob {
  id: string;              // Unique job UUID
  token: string;           // 3-digit student print token (e.g., PRN
  fileName: string;        // Original uploaded name
  fileSize: number;        // File size in bytes
  pageCount: number;       // Calculated total page count
  copies: number;          // Quantity requested
  printMode: 'mono'|'color'; // Monochrome vs vivid color choice
  sides: 'single'|'double'; // Simplex vs Duplex long-edge flip
  pageRange?: string;      // Custom pages (e.g., "1-3, 5")
  status: 'queued' | 'printing' | 'completed';
  studentName: string;
  studentEmail: string;
  createdAt: string;
  progressPercent: number;
  serverFilePath: string;  // Local path inside server uploads folde
  reason?: string;
  scheduledFor?: string;
  shopId: string;          // Tenant print center identifier
}

export interface Shop {
  id: string;              // Unique shop ID (e.g. alliance_print)
  name: string;            // Display name
  phone: string;           // Customer support number
  address: string;         // Campus location details
  isOpen: boolean;         // Open for immediate print orders
  openingTime: string;     // Time formatting (e.g. 08:00 AM)
  closingTime: string;     // Time formatting (e.g. 08:00 PM)
}
```

- **Reading/Writing Transactions**:

```
export function readDb(): Db {
  ensureDir();
  if (!fs.existsSync(DB_PATH)) return { jobs: [], shops: DEFAULT_SHO
  try {
    const data = JSON.parse(fs.readFileSync(DB_PATH, 'utf-8'));
    if (!data.shops) data.shops = DEFAULT_SHOPS;
    return data;
  } catch {
    return { jobs: [], shops: DEFAULT_SHOPS };
  }
}

export function writeDb(db: Db): void {
  ensureDir();
  fs.writeFileSync(DB_PATH, JSON.stringify(db, null, 2));
}
```

---

4. The Backend Server & Security Validations (`server/index.ts`)

This runs on Port `3001` and implements the REST API and security checks.

1. Ingestion File validation (MIMEs and Extensions)

To prevent users from uploading unapproved files, we list allowed formats:

```
const allowedExts = ['.pdf', '.png', '.jpg', '.jpeg', '.doc', '.docx', '.ppt
const allowedMimes = [
  'application/pdf',
  'image/png',
  'image/jpeg',
  'image/jpg',
  'image/pjpeg',
  'application/msword',
  'application/vnd.openxmlformats-officedocument.wordprocessingml.document',
  'application/vnd.ms-powerpoint',
  'application/vnd.openxmlformats-officedocument.presentationml.presentation
];
```

2. Magic Bytes Signature Checks

Renaming a file extension (e.g., renaming a malicious `.exe` file to `invoice.docx`) bypasses simple extension checks.

Our server solves this by reading the **first 8 bytes** of the file to verify its true binary signature (header):

```
const buffer = Buffer.alloc(8);
const fd = fs.openSync(file.path, 'r');
fs.readSync(fd, buffer, 0, 8, 0); // Read the first 8 bytes
fs.closeSync(fd);
const hex = buffer.toString('hex').toUpperCase();

if (ext === '.pdf') {
  isSignatureValid = hex.startsWith('25504446'); // '%PDF' header in hexadec
} else if (ext === '.png') {
  isSignatureValid = hex.startsWith('89504E47'); // '\x89PNG' header
} else if (ext === '.jpg' || ext === '.jpeg') {
  isSignatureValid = hex.startsWith('FFD8FF');   // JPEG SOI marker
} else if (ext === '.docx' || ext === '.pptx') {
  isSignatureValid = hex.startsWith('504B0304'); // 'PK\x03\x04' ZIP file co
} else if (ext === '.doc' || ext === '.ppt') {
  isSignatureValid = hex.startsWith('D0CF11E0'); // Compound File Binary For
}
```

If a user tries to upload an executable renamed to `.docx`, the file is blocked because an `.exe` file starts with `4D5A` (MZ header), which doesn't match the Office ZIP format signature.

5. The Frontend Client Console & State Persistence (`StudentPortal.tsx`)

This React component implements the **Print Deck Console** UI and persistent browser sessions.

1. Persistent Authentication Flow (`localStorage`)

In Python or C++, program memory resets when the script ends. In browsers, we can save data permanently across restarts using the `localStorage` **Key-Value store** (which does not have an expiration date). This keeps the user logged in even after two months.

- **Initialization check on load**: We check browser storage for an active session:

```
const [studentName, setStudentName] = useState(() => localStorage.ge
const [studentEmail, setStudentEmail] = useState(() => localStorage.
const [isRemembered, setIsRemembered] = useState(() => !!(localStora
```

- **Dual Sign-In Paths**:
- **Username & Password**: Authenticates the credentials and derives the student profile from the username, writing the session to `localStorage` permanently.
- **Google Sign-In**: Opens a high-fidelity modal dialog mimicking Google's account chooser, saving the profile name and email on confirmation.
- **The User Flow**:
- If `isRemembered` is true: The Sign-In screen is completely bypassed. The interface displays an active profile card (e.g. "Basav (basav@university.edu)") with a **"SIGN OUT"** button, and immediately displays the file upload controls.
- If `isRemembered` is false: The file upload controls and LCD display are hidden. The user is presented with the credentials card containing Google and Username inputs.

2. Previews & Object URLs

We convert in-memory files to temporary browser URLs:

```
updatedUrls[file.name] = URL.createObjectURL(file);
```

On unmount or form reset, we clean them up using `URL.revokeObjectURL(url)` to prevent memory leaks.

- **PDF Previews**: Rendered natively inside an